Application Flow Chart

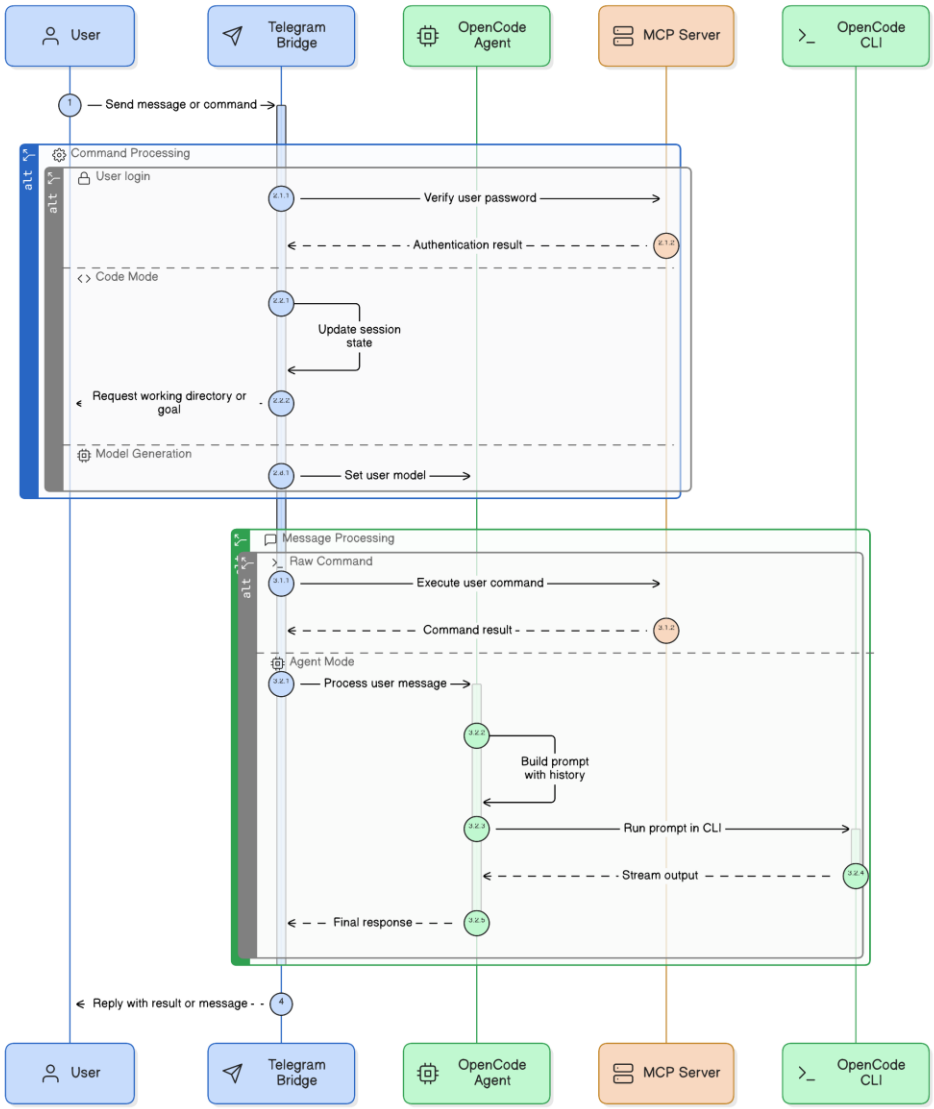
This document outlines the architecture and data flow of the Remote MCP Control System.

Architecture Overview

The system consists of three main components orchestrated by the `RemoteAgentApp`:

1. **TelegramBridge**: Handles user interaction via Telegram.
2. **OpenCodeAgent**: Processes natural language requests using the `opencode` CLI.
3. **MCPServer**: specific tools and raw command execution.

Message Processing Flow



detailed Component Interaction

1. Initialization (`src/main.py`)

- `RemoteAgentApp` initializes `MCPServer`.
- `RemoteAgentApp` initializes `OpenCodeAgent`.
- `RemoteAgentApp` initializes `TelegramBridge` injecting `mcp_server` and `opencode_agent`.

2. Telegram Bridge (`src/bridges/telegram_bridge.py`)

- Acts as the entry point for all user interactions.
- Manages **User Sessions**:
  - Tracks authentication state.
  - specialized modes (e.g., `CODING` mode).
  - Session history and context.
- Routes messages:
  - **Commands** (`/start`, `/login`): Handled locally or via MCP Auth.
  - **Raw Commands**: Routed to `MCPServer`.
  - **Natural Language**: Routed to `OpenCodeAgent`.

3. OpenCode Agent (`src/core/opencode_agent.py`)

- Wraps the `opencode` CLI.
- Manages **User Context** (history of recent messages) to build context-aware prompts.
- **Direct File Search**: Optimized checking for file requests before calling the heavy agent.
- executes the request via `opencode run` and streams the output back.

4. MCP Server (`src/core/mcp_server.py`)

- Provides the core toolset (`ShellTool`, `FilesTool`, etc.).
- Used by `TelegramBridge` for raw command execution and authentication checks.
- Can be exposed as a standalone MCP server.